```
var a = 1;
let b = 2;
const c = 3;

{
    console.log("Inside block");
    var a = 10;
    let b = 20;
    const c = 30;
    d = 40;

    console.log("a = ",a); //
    console.log("b = ",b);
    console.log("c = ",c);
    console.log("d = ",d);
}

    console.log("Outside block");

    console.log("a = ",a);
    console.log("b = ",b);
    console.log("c = ",c);
    console.log("d = ",d);
```

Explanation of above code according to GEC.

- GEC will run this code in two phase.
    1. Variable phase.
    2. Execution phase.
- When the control enter to the block scope then the starting from the bracket, variable phase will firstly search for var type variable then he search same var_name will decleared outside also or not if happen then it will create a space or otherwise point the same data.
- and in the variable phase they store undefiened for the var and the keep empty for both let and const.
- in the execution phase value of the variable will be assigned to the variable and allocates to the memory.
- In the execution phase when they enter into the block scope then firstly search for the let and const type of variable then hr creates a block scope otherwise they only update the value of var in the global scope.
- and if they found let and const in block scope then they again run the variable phase for the block scope only.
- and then block scope will be create and will keep empty because they dont assign any value in variable phase.
- After this execution phase will run for the entire code from the where he stopped.
- In the execution phase when we try to assign the value then they firstly they search that variable in own scope(BLOCK SCOPE), and if they dont found then he search in own parent(script scope) and if they

dont found also then he goes to global scope and update the value of that variable. (This is called **Lexical scope**)

- In the execution phase, when js engine will exit from block scope then they clear the block scope memory using by using garbage collector features.
- Autoglobals will create during execution phase.

Variable shadowing.

var a = 10;

let a = 10; { const a = 1000; c.log("a : ",window.a); }

- If we want to access the outside variable value in inside block and also in inside bock have a same name variable.
- to handle this ., In initial stage window, self, this, frames.

Hoisting.

- The ability of js engine to access a variable before its declaration statement is known as hoisting.
- Variable declared with var, let, and const keyword support hoisting.

What is Temporal Dead Zone?

- It is the time frame between variable declaration and variable initialization. In this time frame we can not access a variable.
- Variable declared with `let` and `const` belongs to temporal dead zone (TDZ).
- If we access value of that variable before actual value to be initialized this phase is called TDZ.

Variable Phases and Behavior

Variable Phase vs Execution Phase

Declaration Type	Variable Phase	Execution Phase
`var`	1. Memory create 2. Store undefined (Value)	1. Actual value initialized during variable phase
`let`	1. Memory create 2. Store undefined (Value)	1. Actual value initialized during Execution phase
`const`	1. Memory create	1. Actual value initialized during Execution phase

---

Detailed Explanation

1. `var` Declaration

- **Variable Phase:**
  - Memory is created
  - Value is stored as `undefined`
- **Execution Phase:**
  - Actual value is initialized during variable phase
  - Can be accessed before initialization (returns `undefined`)

2. `let` Declaration

- **Variable Phase:**
  - Memory is created
  - Value is stored as `undefined`
- **Execution Phase:**
  - Actual value is initialized during execution phase
  - Cannot be accessed before initialization (throws `ReferenceError`)
  - Subject to TDZ

3. `const` Declaration

- **Variable Phase:**
  - Memory is created
- **Execution Phase:**
  - Actual value is initialized during execution phase
  - Cannot be accessed before initialization (throws `ReferenceError`)
  - Subject to TDZ
  - Must be initialized at declaration